LLMを用いた初期集団生成による Evolving Exact Decompilationの探索性能改善

齊藤 遼太[1,a)]　西垣 正勝[1]　大木 哲史[1,2]

概要：Byte-Equivalent Decompilation（BED）は，あるバイナリに対して，再コンパイルによって同一の命令列を生成するソースコードを復元することを目的とする．Evolving Exact Decompilation（EED）は，遺伝的アルゴリズムを用いて BED を実現する手法の一つとして提案されているが，その探索性能は初期集団の品質に依存する．本研究では，大規模言語モデル（LLM）を EED の探索における初期集団の生成器として利用することで，探索性能を改善できるかを検証する．この目的のため，EED 型のシステムを実装し，初期集団の半数を LLM が生成した候補で置き換える LLM-mixed seeding を導入した．先行研究を基に構成した小規模 C プログラムを対象とした実験では，LLM-mixed seeding により，探索終了時の正規化命令トークン類似度が，データベースをもとに初期集団を生成するベースラインと比較して 17.90 パーセントポイント向上した．さらに，初期類似度および実験条件を考慮した分析においても，LLM-mixed seeding では，初期類似度から 100%までに残された差のうち，探索によって縮小された割合が大きかった．これらの結果は，LLM 生成候補を含む初期集団が，再コンパイル結果に基づく進化的探索の有効な出発点となり得ることを示唆する．

キーワード：デコンパイル，リバースエンジニアリング，LLM, 遺伝的探索

Enhancing Evolving Exact Decompilation via LLM-Mixed Seeding

Ryota Saito[1,a)]　Masakatsu Nishigaki[1]　Tetsushi Ohki[1,2]

Abstract: Byte-Equivalent Decompilation (BED) aims to recover source code that, when recompiled, produces an instruction sequence identical to that of a given binary. Evolving Exact Decompilation (EED) has been proposed as an approach to BED based on genetic algorithms, but its search performance depends on the quality of the initial population. In this study, we investigate whether large language models (LLMs) can improve the search performance of EED by using them to generate candidates for the initial population. To this end, we implemented an EED-style system and introduced LLM-mixed seeding, in which half of the initial population is replaced with candidates generated by an LLM. Experiments on small C programs constructed based on prior work showed that LLM-mixed seeding improved the normalized instruction-token similarity at the end of the search by 17.90 percentage points compared with a baseline that generates the initial population from a database. Furthermore, after controlling for the similarity at the start of the search and other experimental conditions, trials using LLM-mixed seeding reduced a larger proportion of the remaining gap to 100% assembly instruction-token similarity between the input and the final output. These results suggest that LLM-generated candidates can provide useful starting points for evolutionary search based on recompilation results.

Keywords: Decompilation, Reverse Engineering, LLM, Evolutionary Search

[1] 静岡大学
Shizuoka University
[2] 理研 AIP
RIKEN AIP
[a)] saito@sec.shizuoka.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

[*1]デコンパイルは，バイナリ実行ファイルから高水準のプログラム表現を復元する技術であり，マルウェア解析や脆弱性調査，バイナリパッチングなどのリバースエンジニアリングで利用されている．一般的なデコンパイラは，人間が理解しやすい擬似コードの生成を主な目的としており，出力をそのまま再コンパイルすることは困難である．また，復元されたプログラムには，制御フローやデータ型，データ構造などに誤りが含まれる場合がある．

Byte-Equivalent Decompilation（BED）は，これとは異なり，復元したソースコードを再コンパイルした際に，入力バイナリと等価な命令列を生成することを目的とする．Evolving Exact Decompilation（EED）[1] は，BED を進化的探索として実現する手法である．EED では，ソースコードとアセンブリ命令の組からなるデータベースから初期候補を取得し，候補ソースコードをコンパイルして入力バイナリとの類似度を評価しながら，遺伝的アルゴリズムによって候補を更新する．

先行研究によれば，EED では小規模な関数であってもバイト等価な候補の発見までに数万回の反復を要することが報告されている [1]．EED の遺伝的探索は，データベースから取得した候補を初期集団として開始するため，初期集団の品質は有限の探索予算下での到達点に影響する．一方，データベースから取得した初期候補とターゲット命令列との類似度は最大 55%にとどまることが報告されており [1]，より優れた初期候補生成アルゴリズムを採用することで探索を改善できる可能性がある．

近年では LLM4Decompile [2] をはじめ，大規模言語モデル（Large Language Model; LLM）を用いてアセンブリコードからソースコードを生成するデコンパイル手法が提案されている．しかし，これらの手法は生成結果と入力バイナリとのバイト等価性を保証するものではない．そこで本研究では，LLM の出力を最終的なデコンパイル結果として用いるのではなく，EED の初期候補として利用する．LLM による候補生成と，再コンパイル結果に基づく進化的探索を組み合わせることで，バイト等価なソースコードの探索を改善できるかを検証する．

本研究では，EED 型の進化的探索を実装し，初期集団の半数を LLM 生成候補に置き換える LLM-mixed seeding を導入する．オリジナルの EED で用いられたタスクをもとに再構築した小規模な単一関数 C プログラムを対象とし，初期集団をすべてデータベースから生成するベースラインと比較する．両構成で進化的操作，コンパイラ設定，正規化手法，評価予算を揃え，LLM-mixed seeding が最終類似度および初期化後の探索進捗に与える影響を評価する．実験の結果，LLM-mixed seeding は，固定された探索予算のもとで最終的な正規化命令トークン類似度を平均 17.90 ポイント改善した．さらに，この差は初期集団の類似度が高いことだけでは説明されず，初期類似度および実験条件を考慮した場合でも，LLM-mixed seeding では 100%の類似度までに残された差をより大きく縮小した．

本研究の貢献は以下の二点である．

- EED の初期集団の一部を LLM 生成候補によって構成する LLM-mixed seeding を導入し，LLM を最終出力ではなく進化的探索の初期化に利用する方法を提案した．
- 探索予算を固定した実験により，LLM-mixed seeding が最終命令トークン類似度を改善し，さらに初期類似度を考慮した場合にも，その後の進化的探索による進捗が大きくなることを示した．

2. 関連研究

2.1 プログラム解析に基づくデコンパイル

デコンパイルでは，バイナリプログラムから高水準かつ人間が理解可能な表現を復元するための手法が研究されてきた．Cifuentes らは，デコンパイルを制御フローの復元，データフロー解析，高水準プログラム構造の再構成からなるプログラム解析問題として体系化した [3]．その後，ネイティブバイナリを対象として，意味を保存しながら制御フローを高水準の構造へ復元する手法が提案されている [4]．また，既存デコンパイラが生成するコードと元のソースコードとの差異を体系的に分析する研究も行われている [5]．これらの研究は，人間による解析に利用可能な高水準表現の復元を主な対象としており，再コンパイルによって対象バイナリの命令列を再現することを直接の目的とはしていない．

2.2 Byte-Equivalent Decompilation と Evolving Exact Decompilation

Evolving Exact Decompilation（EED）は，Byte-Equivalent Decompilation（BED）を探索問題として扱う手法である [1]．EED では，候補ソースプログラムを再コンパイルし，生成された命令列と対象バイナリとの類似度を評価しながら，遺伝的アルゴリズムによって候補を反復的に改良する．初期集団には，ソースコードデータベースから取得した候補や既存デコンパイラの出力を利用する．

本研究では，EED における初期集団の生成に LLM を導入し，LLM 生成候補とデータベースから取得した候補を組み合わせる LLM-mixed seeding を用いる．これにより，探索手続きと探索予算を共通とした条件下で，初期集団の生成方法が EED 型探索の到達性能に与える影響を評価する．

[*1] 本稿は，SURE2026（2026 年 11 月 19 日）に採択された国際会議論文を基に，日本語による説明の再構成および内容の補足を行ったものである．

© 2026 Information Processing Society of Japan

2.3 機械学習・LLM を用いたデコンパイル

機械学習を用いたデコンパイルも研究されており，Coda はデコンパイルを End-to-End のニューラル翻訳問題として扱う手法を提案した [6]．近年では，SLaDe [7] や LLM4Decompile [2] のように，言語モデルを用いてアセンブリコードからソースコードを生成する手法が提案されている．また，DeGPT は既存デコンパイラの出力を LLM によって修正する手法を提案している [8]．

これらの手法では，機械学習モデルや LLM によって生成または修正されたコードをデコンパイル結果として利用する．これに対して本研究では，LLM の出力自体を最終的な復元結果として評価するのではなく，データベースから取得した候補とともに EED 型探索の初期集団を構成するために利用する．その後は，再コンパイル結果に基づく適応度評価と進化的操作によって候補を改良し，LLM 生成候補を導入した初期化が固定された探索予算下での到達性能に与える影響を評価する．

3. 提案手法

3.1 概要

図 1 に提案手法の概要を示す．入力としてターゲットバイナリを与え，探索時の比較対象となる命令列を抽出する．本手法の目的は，再コンパイルによってターゲットと一致する命令列を生成する C ソースプログラムを探索することである．人間が理解可能なソースコードの復元を主目的とする一般的なデコンパイラとは異なり，本手法ではデコンパイルを候補ソースプログラムに対する探索問題として扱う．初期集団の生成後は，EED に基づく進化的探索によって候補を反復的に更新し，再コンパイル後の命令列がターゲット命令列に一致，または十分に近い候補を探索する．

本研究では，この初期化処理に LLM-mixed seeding を導入する．先行研究の実装のようにデータベースから取得した候補のみで初期集団を構成するのではなく，初期集団の一部を LLM が生成した候補に置き換えることで，探索開始時点の候補品質を改善し，限られた評価回数の中でより高い類似度に到達することを目指す．LLM の出力は最終的なデコンパイル結果として用いるのではなく，進化的探索の初期候補としてのみ利用する．

3.2 探索目的と適応度関数

本研究では，再コンパイルによってターゲットバイナリと一致する命令列を生成する C ソースプログラムを探索する．ターゲットと候補の比較には，生のバイト列ではなく正規化した命令トークン列を用いる．各命令トークンは，逆アセンブルされた 1 命令について，ニーモニックと正規化したオペランドを組み合わせたものである．

比較に先立ち，ターゲットバイナリおよび各候補をコンパイルして得られたバイナリを逆アセンブルし，直接分岐および直接呼出しのターゲットを PC 相対変位に，既知のセクションを参照するアドレスをセクション相対オフセットに書き換えるとともに，パディング命令を除去する．これにより，コード配置に起因する差異を抑えつつ，命令列の構造を比較できるようにする．

この正規化によって得られた命令トークン列を用いて，候補の適応度および実験結果の類似度を算出する．ターゲットバイナリから得られた正規化命令トークン列を $t = \langle t_1, \ldots, t_m \rangle$，候補ソースプログラムをコンパイルおよび逆アセンブルして得られた正規化命令トークン列を $c = \langle c_1, \ldots, c_n \rangle$ とする．進化的探索では，先行研究と同様に，ターゲット命令列 t と候補命令列 c の Levenshtein 距離に基づいて適応度を評価する [1]．Tournament selection では命令列全体の編集距離をスカラーの適応度として用いる一方，lexicase selection では，編集距離の計算に基づいて命令列を差分領域に分割し，各領域の編集距離から適応度ベクトルを構成する．この差分領域は，後述する突然変異の対象位置を決定する際にも利用する．

一方，実験結果の報告においては進化的探索と異なり，LCS に基づく類似度を用いる．これは，ベンチマークごとに命令列長が異なるため，絶対的な編集距離では異なるターゲット間の候補品質を直接比較しにくいためである．類似度は命令列長を考慮して次式で算出する．

$$\mathrm{sim}(t, c) = \frac{2 \cdot \mathrm{LCS}(t, c)}{|t| + |c|} \times 100,$$

ここで，$\mathrm{LCS}(t, c)$ は t と c の最長共通部分列の長さを表す．この類似度は 0 から 100 の値をとり，異なるターゲット間で候補品質を共通の尺度によって比較できる．Levenshtein 距離が 0 となることと，LCS に基づく類似度が 100 となることはいずれも $t = c$ と同値であるため，探索における完全一致の停止条件と実験結果における類似度 100 の判定は一致する．

3.3 初期集団生成

進化的探索を開始する前に，探索の初期集団を構成する C ソースプログラム（以下，初期候補）を生成する．ベースラインでは，初期集団のすべてをデータベースから取得した候補によって構成する．一方，LLM-mixed seeding では，設定した割合の初期候補を LLM によって生成し，残りをデータベースから取得した候補によって構成する．

まず，各ターゲット関数について，固定された関数シグネチャとスタブ本体を持つ C コードのテンプレートを作成する．データベースまたは LLM から得られた候補関数の本体をこのテンプレートに挿入することで，関数の周辺にあるコンパイル条件を固定したまま，関数本体を探索対象とする．本研究では，ターゲット関数およびそのシグネチャは再構成したベンチマークから与え，関数境界の検出，シグネチャ推定，型復元は EED 型探索とは別のフロント

© 2026 Information Processing Society of Japan

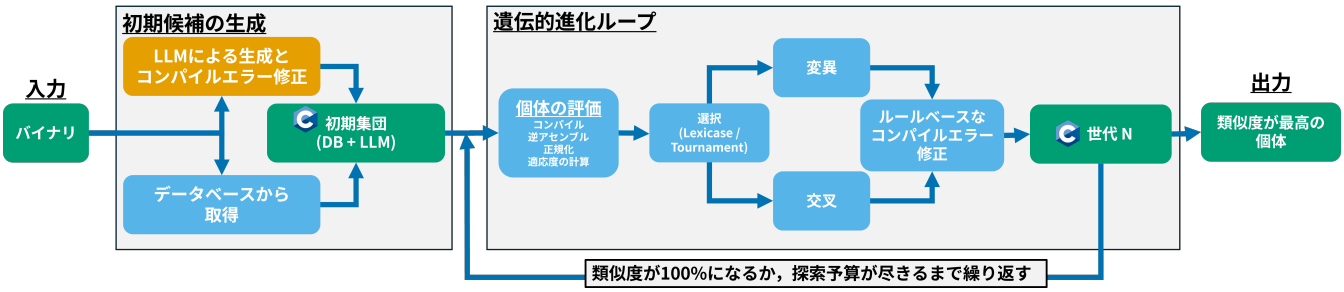


図 1 提案手法の概要
Fig. 1 Overview of the proposed method.

エンド上の課題として扱う．

データベースを用いる場合，各ターゲット関数について，正規化した逆アセンブル結果および関数名を検索情報として候補関数を取得する．取得された関数の中から，類似度を考慮したサンプリングによって候補を選択し，その関数本体をターゲットのテンプレートに挿入する．先行研究で使用された Project Euler コーパスおよび Pliny データベース基盤は公開されていないため，本実装では代替コーパスとして Project CodeNet を使用し [9]，SQLite を用いて検索機構を実装する．

LLM による初期候補の生成も関数単位で行う．LLM には正規化前の逆アセンブル結果を入力し，適応度計算に用いる正規化命令トークン列，ターゲット関数のシグネチャ，および C コードのテンプレートは与えない．生成結果を Tree-sitter で解析し，対象関数名と一致する関数定義から関数本体を抽出してターゲットのテンプレートへ挿入する．利用可能な関数本体を抽出できなかった場合には，テンプレートのスタブを残す．生成した初期候補がコンパイルできない場合には，まずルールベースの修正を行う．LLM 生成候補について，修正後もコンパイルできない場合には，LLM によるコンパイルエラー修正を適用する．

3.4 進化的探索

初期集団の生成後，固定サイズの候補集団を世代ごとに更新することで探索を行う．各世代では，すべての候補をコンパイル，逆アセンブル，正規化し， 3.2 で述べた適応度によって評価する．評価済みの候補から設定された選択手法に従って親個体を選択し，交叉および突然変異によって次世代を生成する．探索は，ターゲットと一致する候補が得られるか，あらかじめ設定した評価回数の上限に達した時点で終了する．

進化的探索中に生成された候補がコンパイルに失敗した場合には，試行回数を制限したルールベースの修正を行う．設定された回数の修正を行ってもコンパイルできない候補には，最悪の適応度を与える．この修正処理は候補評価の一部として実行するものであり，遺伝的アルゴリズムにおける突然変異とは区別される．また，文字列リテラルおよび数値リテラルについては，ターゲットバイナリから取得した値への置換によって適応度が改善する場合に，リテラルを置換する．

親個体の選択手法として，先行研究と同様に tournament selection と lexicase selection を用いる [1]．Tournament selection では正規化命令トークン列間の Levenshtein 距離を用いて候補を比較し，lexicase selection では 3.2 で述べた領域ごとの適応度ベクトルを用いる．交叉には homologous crossover を用い，両親の命令列の対応関係に基づいて，対応するソースコードの範囲を組み合わせる．

突然変異は，ソースコードの文字列ではなく AST 上の文を単位として行う．先行研究で用いられた cut，insert，swap，replace の 4 種類の突然変異演算子を実装する [1]．Cut は選択した文を削除し，insert は現在の候補またはデータベースから取得した文を選択位置の前に挿入する．Swap は同一ブロック内の二つの文の位置を交換し，replace は選択した文を現在の候補またはデータベースから取得した文で置換する．また， 3.2 で求めた差分領域をソースコード上の文に対応付けることで，ターゲット命令列と一致していない領域に対応する文を突然変異の対象として選択できるようにする．

4. 実験と評価

提案手法である LLM-mixed seeding の有効性を評価するため，データベースから取得した候補のみで初期集団を構成するベースラインと比較実験を行う．両集団においては初期集団の生成方法のみを変更し，以下の二つのリサーチクエスチョンを設定する．

- **RQ1**：評価回数の上限を同一とした場合，LLM-mixed seeding はベースラインと比較して，探索終了時に得られる類似度を改善するか．
- **RQ2**： 初期集団の品質を考慮した場合でも，LLM-mixed seeding は，100%の類似度までに残された差を，進化的探索によってベースラインより大きく縮小するか．

RQ1 では，初期集団の生成方法を変更することによって，探索終了時に得られる候補の品質がどの程度変化する

© 2026 Information Processing Society of Japan

かを評価する．一方 RQ2 では，RQ1 で観測される差が，LLM によって初期集団から類似度の高い候補が得られたことだけに起因するのか，もしくはその後の進化的探索における改善にも差が生じているのかを検証する．

4.1 実験設定

4.1.1 比較条件

本実験では，初期集団の生成方法が異なる二条件を比較する．ベースラインでは全候補をデータベースから取得し，LLM-mixed seeding では半数を decompilation 向けコードモデル `enhanced-llm4decompile-end-6.7b` が生成し，残り半数をデータベースから取得する．LLM 生成候補には 3.3 で述べたコンパイルエラー修正を適用する．そのうち LLM による修正には `gpt-4.1-mini` を用い，データベース由来の初期候補および進化的探索で生成された候補には適用しない．初期集団生成後の進化的探索は両条件で共通とする．したがって，本実験では LLM 生成候補単体ではなく，候補生成と初期段階の LLM 修正を含む LLM-mixed seeding 全体の効果を評価する．

4.1.2 評価対象と実験因子

評価対象は，先行研究 [1] と同様に，*Learn C the Hard Way* および Project Euler に由来する小規模 C プログラムとする．ただし，先行研究で使用されたソースコードそのものは公開されていないため，本研究では各演習および問題文に基づいて対象プログラムを再実装した．したがって，本実験では先行研究と同一の課題を扱うが，ソースコードおよび生成されるバイナリが先行研究のものと一致することは保証されない．

先行研究で扱われた 19 プログラムのうち，現在の実装では単一関数のみを探索対象としていることから，複数関数を含む 1 プログラムを除外し，18 プログラムを評価対象とする．これらの各プログラムに対し，コンパイラ最適化レベル $\mathrm{opt} \in \{\mathrm{O0}, \mathrm{O2}\}$ と，親個体の選択方法 $\mathrm{sel} \in \{\mathrm{lexicase}, \mathrm{tournament}\}$ を組み合わせて実験条件を構成する．この組合せにより 72 条件を設定し，各条件について 3 回の独立した試行を行うため，各比較条件につき合計 216 試行となる．各プログラムを対応するコンパイル条件で生成したバイナリを，探索のターゲットとして使用する．

4.1.3 探索条件

初期集団の生成方法以外の探索条件は，二つの比較条件で統一する．集団サイズは 256 とし，1 試行あたりの候補評価回数の上限を 32,768 回とする．また，各候補に対するルールベースのコンパイルエラー修正は最大 3 回までとする．探索は，100%の類似度を持つ候補が得られた場合，もしくは候補評価回数が 32,768 回に達した場合に終了する．

4.1.4 コンパイラ設定

ターゲットバイナリと探索中の候補は，いずれも `Clang 14.0.6` を用いて x86-64 向けにコンパイルする．共通のコンパイルオプションとして`-m64 -g -fno-pie -no-pie` を指定し，最適化レベルのみ`-O0` または`-O2` に変更する．したがって，各条件では以下のいずれかのコマンドを使用する．

```
clang -m64 -g -O0 -fno-pie -no-pie
clang -m64 -g -O2 -fno-pie -no-pie
```

4.1.5 評価指標

各試行について，進化的探索を開始する前の初期集団に含まれる候補のうち，最も高い LCS ベース類似度を**初期最高類似度** S_0 とする．また，探索終了時までに得られた候補のうち，最も高い類似度を**最終到達類似度** S_f とする．S_0 は探索開始時点における候補の品質を表し，S_f は所定の評価回数内で探索が到達した候補の品質を表す．さらに，初期集団の品質とその後の進化的探索による改善を区別するため，各試行について残存差分縮小率を

$$\frac{S_f - S_0}{100 - S_0}$$

と定義する．残存差分縮小率は，探索開始時点で 100%までに残されていた差のうち，進化的探索によって縮小された割合を表す．例えば，類似度が 40%から 70%に改善した場合と，80%から 90%に改善した場合では，いずれも 100%までの差を半分縮小しているため，残存差分縮小率は 50%となる．

なお，$S_0 = 100$ の場合には分母が 0 となるため，残存差分縮小率を定義できない．したがって，初期集団の時点ですでに 100%の類似度に到達した試行は，RQ2 における残存差分縮小率の分析から除外する．

4.2 RQ1：固定された評価回数における最終類似度

RQ1 では，同一の評価回数上限のもとで，LLM-mixed seeding によって探索終了時の類似度が改善するかを評価する．各ベンチマーク，最適化レベル，選択方法の組合せについて 3 試行の最終類似度の平均を求め，LLM-mixed seeding とベースラインとの差を

$$\Delta = \mathrm{final}_{\mathrm{LLM}} - \mathrm{final}_{\mathrm{base}}$$

とし，Δ の単位はパーセントポイント（pp）とする．同一のベンチマークから最適化レベルと選択方法の異なる複数の条件が得られるため，統計的検定ではこれらを独立した観測値として扱わず，全体の比較では，各ベンチマークについて 4 つの最適化レベル・選択方法の組合せにおける Δ を平均し，18 ベンチマークを観測単位として Wilcoxon の符号付順位検定を行う．また，各最適化レベル・選択方法の組合せについても，各ベンチマークが一つの観測値となるよう 18 ベンチマークを対応付けて検定する．

表 1 に結果を示す．全 18 ベンチマークでは，LLM-mixed seeding の最終類似度がベースラインを平均 17.90 pp 上回

© 2026 Information Processing Society of Japan

表 1 ベンチマークごとの平均最終類似度の差
Table 1 Difference in Mean Final Similarity by Benchmark.

条件	n	平均 Δ	中央値	p 値
全体	18	+17.90	+16.48	7.6×10^{-5}
O0 / lexicase	18	+12.48	+7.11	1.6×10^{-3}
O0 / tournament	18	+11.15	+2.30	2.0×10^{-2}
O2 / lexicase	18	+26.20	+19.56	1.5×10^{-5}
O2 / tournament	18	+21.76	+15.10	3.3×10^{-4}

表 2 最適化レベルごとの類似度閾値への到達率
Table 2 Similarity Threshold Attainment by Optimization Level.

	O0		O2	
閾値	Base	LLM	Base	LLM
$\geq 50\%$	59.3%	66.7%	18.5%	63.9%
$\geq 60\%$	31.5%	51.9%	5.6%	47.2%
$\geq 70\%$	14.8%	47.2%	5.6%	34.3%
$\geq 80\%$	5.6%	38.9%	3.7%	17.6%
$\geq 90\%$	0.0%	21.3%	0.0%	11.1%
100%	0.0%	11.1%	0.0%	6.5%

り，中央値は+16.48 pp であった．ベンチマーク単位の Wilcoxon の符号付順位検定では，$p = 7.6 \times 10^{-5}$ となった．また，最適化レベルと選択方法の 4 つの組合せのいずれにおいても，LLM-mixed seeding の平均最終類似度はベースラインを上回った．

最適化レベル別にみると，O0 では lexicase selection で+12.48 pp，tournament selection で+11.15 pp であったのに対し，O2 ではそれぞれ+26.20 pp，+21.76 pp であり，いずれの選択方法でも O2 の方が大きな改善幅を示した．ただし，本実験から直接確認できるのは，O2 において改善幅が大きかったことまでであり，その要因をこの結果のみから特定することはできない．

類似度の閾値ごとの到達率を**表 2** に示す．O0 と O2 のいずれにおいても，すべての閾値で LLM-mixed seeding の到達率がベースラインを上回った．例えば O2 では，60%以上の類似度に到達した試行が，ベースラインでは 5.6%であったのに対し，LLM-mixed seeding では 47.2%であった．また，ベースラインでは 100%に到達した試行がなかった一方，LLM-mixed seeding では O0 で 11.1%，O2 で 6.5%の試行が 100%に到達した．

以上より，ベンチマークを観測単位として比較した場合にも，固定された評価回数のもとで LLM-mixed seeding はベースラインより高い最終類似度に到達した．ただし，この差には初期集団の時点で得られる候補の品質の違いも含まれる．RQ2 ではこの影響を考慮し，初期集団の生成後に進化的探索がどの程度の改善をもたらしたかを評価する．

4.3 RQ2：初期類似度を考慮した探索進捗

RQ1 で確認した最終類似度の差には，LLM-mixed seeding による初期候補の改善と，その後の進化的探索による改善の双方が含まれる．したがって，最終類似度の比較だけでは，LLM-mixed seeding の効果が初期候補の改善だけによるものか，もしくは進化的探索による改善にも差が生じているのかを判別できない．そこで RQ2 では，初期集団の品質を考慮した上で，進化的探索による改善を比較する．

まず，RQ1 で得られた最終類似度の差を，初期類似度の差と，初期化後の探索による改善量の差に単純に分解する．各試行における探索による改善量を $G = S_f - S_0$ とし，各条件について 3 試行の平均を求める．LLM-mixed seeding とベースラインの条件間差をそれぞれ

$$\Delta_{\text{seed}} = \bar{S}_{0,\text{LLM}} - \bar{S}_{0,\text{base}}, \qquad \Delta_{\text{search}} = \bar{G}_{\text{LLM}} - \bar{G}_{\text{base}}$$

と定義する．このとき，RQ1 で用いた最終類似度の差は $\Delta = \Delta_{\text{seed}} + \Delta_{\text{search}}$ と表せる．以上の分解を全 72 条件に適用すると，最終類似度の平均差+17.90 pp に対し，初期最高類似度の差 Δ_{seed} は+17.29 pp，探索による改善量の差 Δ_{search} は+0.60 pp であった．最適化レベル別では，O0 で $\Delta_{\text{seed}} = +17.26$ pp，$\Delta_{\text{search}} = -5.44$ pp，O2 でそれぞれ+17.33 pp，+6.65 pp であった．この分解では，RQ1 で観測された最終類似度の差の大部分が初期集団の時点で生じているように解釈できるが，用いられる G は初期類似度からの絶対的な改善量であるため，初期類似度が高いほど 100%までの改善可能な幅が小さくなる影響を受ける．そこで，4.1.5 で定義した残存差分縮小率を用いて，初期類似度の違いを考慮した探索進捗を比較する．なお，初期集団の時点で類似度 100%に到達していた LLM-mixed seeding の 15 試行は，残存差分縮小率を定義できないため分析から除外した．

表 3 初期類似度ごとの残存差分縮小率
Table 3 Residual Closure by Initial-Similarity Band.

	Baseline			LLM-mixed		
初期類似度	n	中央値	exact	n	中央値	exact
$[0, 40)$	148	13.3%	0	104	15.1%	0
$[40, 50)$	34	18.7%	0	17	50.9%	0
$[50, 60)$	28	7.2%	0	39	48.1%	1
$[60, 100)$	6	0.0%	0	41	12.4%	3

初期類似度の範囲ごとに残存差分縮小率を集計した結果を**表 3** に示す．初期類似度が 40%以上 50%未満の範囲では，残存差分縮小率の中央値がベースラインの 18.7%に対して LLM-mixed seeding では 50.9%であった．50%以上 60%未満の範囲でも，ベースラインの 7.2%に対して LLM-mixed seeding では 48.1%であり，LLM-mixed seeding の方が大きな改善を示した．なお，表中の exact は，初期集団では類似度 100%未満であり，その後の探索によって 100%に到達した試行数を表す．

一方，初期類似度が 60%以上 100%未満の範囲では，ベースラインが 6 試行であるのに対して LLM-mixed seeding は

© 2026 Information Processing Society of Japan

41 試行であり，両条件の試行数に大きな差がある．このため，この範囲について残存差分縮小率を直接比較することは適切ではない．また，ベースラインではほとんど得られなかった 60%以上の初期類似度を持つ試行が，LLM-mixed seeding では多数得られていることが分かる．

初期類似度の分布の違いを考慮するため，残存差分縮小率を目的変数とし，初期類似度，最適化レベル，選択方法，評価対象プログラムを説明変数とした線形回帰分析を行った．評価対象プログラムを固定効果として含めたモデルでは，LLM-mixed seeding の係数は +0.164 であり，評価対象プログラム単位でクラスタ化した標準誤差を用いた場合にも統計的に有意であった（$p = 2.4 \times 10^{-3}$）．

さらに，LLM-mixed seeding によって初期集団が少数の類似した候補に偏り，探索に利用できる候補の多様性が低下している可能性を検討した．初期集団の多様性を直接評価することは困難であるため，ここではコンパイル可能な候補に含まれる異なる適応度の数を指標として用いる．この値は意味的な多様性そのものを表すものではないが，初期集団が少数の候補に集中しているかを確認するための指標となる．

初期候補のコンパイル成功率は，ベースラインでは O0 で 41.1%，O2 で 33.3%であったのに対し，LLM-mixed seeding ではそれぞれ 57.5%，58.1%であった．また，異なる適応度を持つ候補の数は，LLM-mixed seeding でベースラインのおよそ 3 倍から 7 倍であった．したがって，少なくともこの指標からは，LLM-mixed seeding によって初期集団の多様性が単純に低下したとはいえない．

LLM-mixed seeding では 19 試行が 100%の類似度に到達したのに対し，ベースラインでは 100%に到達した試行はなかった．LLM-mixed seeding の 19 試行のうち 15 試行では，初期集団の時点ですでに 100%の候補が得られていた．一方，残る 4 試行では初期集団の最高類似度は 100%未満であり，その後の進化的探索によって 100%に到達した．

以上より，LLM-mixed seeding は初期集団に類似度の高い候補を導入するだけでなく，初期類似度を考慮した場合にも，その後の進化的探索による改善が大きかった．また，初期集団では 100%に達していなかった候補から進化的探索によって 100%に到達した試行も確認された．これらの結果は，LLM による初期候補生成と EED 型の進化的探索を組み合わせることで，初期候補の品質向上だけでは説明できない探索進捗が得られることを示している．

5. 議論

5.1 RQ1：固定された探索予算における LLM-mixed seeding の効果

RQ1 の結果から，固定された評価回数のもとで，LLM-mixed seeding はベースラインより高い最終到達類似度に到達した．また，最適化レベルと選択方法の四つの組合せのいずれにおいても，ベンチマーク間で平均した改善量は正であった．したがって，本実験の範囲では，LLM 生成候補を初期集団に導入することで，固定された探索予算内での EED 型探索の到達性能が改善したといえる．

4.3 に示す分解では，最終到達類似度の平均差+17.90 pp に対し，初期最高類似度の差は+17.29 pp，探索による改善量の差は+0.60 pp であった．このことから，RQ1 で観測された改善には，LLM 生成候補によってターゲットに近い候補から探索を開始できたことが大きく寄与したと考えられる．一方，改善幅は O0 より O2 で大きかったが，本実験ではその要因を直接評価しておらず，最適化レベルによる差の原因は特定できない．また，個別の実験条件では LLM-mixed seeding がベースラインを下回る場合もあり，すべての評価対象に対する改善を保証する結果ではない．

5.2 RQ2：初期候補導入後の進化的探索による改善

RQ2 では，初期最高類似度および実験条件を考慮した場合にも，LLM-mixed seeding でその後の進化的探索による改善が大きいことが示唆された．評価対象プログラムを固定効果として含む回帰分析では，LLM-mixed seeding の残存差分縮小率に対する係数は+0.164 であり，初期候補の品質だけでは RQ1 で観測された最終到達類似度の差を説明できないことが示された．

また，LLM-mixed seeding で 100%の類似度に到達した 19 試行のうち，4 試行では初期集団の最高類似度が 100%未満であり，その後の進化的探索によって 100%に到達した．この結果は，LLM 生成候補が生成時点でターゲットと完全に一致していない場合でも，後続の探索によって改善可能な開始点となる場合があることを示している．

一方，本分析から，LLM 生成候補のどの性質が後続の進化的探索に寄与したかを特定することはできない．初期集団ではコンパイル成功率や異なる適応度を持つ候補数の増加が確認されたものの，これらは候補の意味的な多様性や進化的操作によって到達可能な探索領域を直接表す指標ではない．したがって，LLM 生成候補の構文構造，交叉や突然変異で利用可能な部分構造，および初期集団における候補分布と探索進捗との関係を今後分析する必要がある．

6. 制約と今後の課題

6.1 評価対象と実装範囲

本研究の評価対象は，先行研究で用いられた課題をもとに再実装した，小規模かつ単一関数の C プログラムに限定されている．実験では，複数の最適化レベル，選択方法および試行を用いたが，評価対象となるプログラム自体の多様性は限定されている．したがって，本研究の結果が，大規模・複数関数のプログラムや異なるコンパイラ，ソースコード領域にも一般化できるとは限らない．また，先行研究で使用された実装および評価対象のソースコードは公開

© 2026 Information Processing Society of Japan

されていないため，本研究の実験結果を先行研究の数値的な再現として解釈することはできない．今後は，より長い関数や異なる種類のプログラムを含む評価対象を用いるとともに，複数のコンパイラおよび最適化条件に対して評価を行い，LLM-mixed seeding の適用範囲を検証する必要がある．加えて，現在の実装は探索対象を単一の関数としており，複数関数を扱うためには，関数境界や呼出し関係，関数シグネチャなどを考慮した探索への拡張が必要である．

6.2 評価指標と探索予算

本研究では，ターゲットと候補を正規化命令トークン列に変換し，その類似度に基づいて探索結果を評価した．したがって，本研究における類似度 100%は，3 章で定義した正規化処理のもとで命令トークン列が一致したことを意味する．これは，実行ファイルを構成するすべてのバイトが一致することを意味せず，また，あらゆる実行環境における意味的等価性を保証するものでもない．このため，本研究で扱う 100%の類似度は，採用した正規化および比較方法の範囲内で解釈する必要がある．また，本実験では探索性能を比較するため，各試行の候補評価回数に同一の上限を設定し，その時点での最終到達類似度を評価した．この方法により同一の評価回数のもとで二つの初期化方法を比較できる一方，実際の計算時間や，100%の類似度へ到達するまでに必要な評価回数を直接評価するものではない．今後は，100%に到達するまでの評価回数や実行時間を測定するとともに，正規化方法の変更が探索結果に与える影響についても検証する必要がある．

6.3 LLM-Mixed Seeding の影響

本研究では，LLM を初期集団の生成に限定して利用した．また，実験で評価した LLM-mixed seeding には，LLM による候補生成だけでなく，コンパイル不能な LLM 生成候補に対する初期段階の LLM 修正も含まれている．したがって，本実験の結果から，LLM による候補生成そのものとコンパイルエラー修正の寄与を個別に評価することはできない．今後は，これらの処理を分離した比較を行い，候補の類似度，コンパイル成功率，初期集団の多様性，および残存差分縮小率への寄与をそれぞれ評価する必要がある．

さらに，本研究では一つの decompilation 向けコードモデルのみを用いたため，異なる decompilation 向けモデルや汎用コードモデルによる比較評価も必要である．

7. 結論

本研究では，EED 型の進化的探索において，初期集団の一部を LLM 生成候補によって構成する LLM-mixed seeding を導入し，固定された探索予算のもとでその探索性能を評価した．先行研究の課題をもとに再構築した小規模な単一関数 C プログラムを対象とした実験では，LLM-mixed seeding により，データベースから取得した候補のみを用いるベースラインと比較して，探索終了時の正規化命令トークン類似度が平均 17.90 pp 向上した．また，初期類似度および実験条件を考慮した分析においても，LLM-mixed seeding では 100%までの残差のうち探索によって縮小された割合が大きく，最終類似度の改善は初期候補の品質向上だけでは説明されなかった．以上の結果から，本実験の範囲では，LLM 生成候補を進化的探索の初期候補として利用することで，EED 型探索の到達性能を改善できることが示された．

8. 謝辞

本研究の一部は JST ムーンショット型研究開発事業 JPMJMS2215 の助成を受けたものです．

参考文献

[1] Eric Schulte, Jason Ruchti, Matt Noonan, et al. Evolving exact decompilation. In *Workshop on Binary Analysis Research (BAR)*, San Diego, CA, USA, February 2018.

[2] Hanzhuo Tan, Qi Luo, Jing Li, et al. LLM4Decompile: Decompiling binary code with large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 3473–3487, Miami, Florida, USA, November 2024. Association for Computational Linguistics.

[3] Cristina Cifuentes and K. John Gough. Decompilation of binary programs. *Software: Practice and Experience*, Vol. 25, No. 7, pp. 811–829, 1995.

[4] David Brumley, JongHyup Lee, Edward J. Schwartz, et al. Native x86 decompilation using Semantics-Preserving structural analysis and iterative Control-Flow structuring. In *22nd USENIX Security Symposium (USENIX Security 13)*, pp. 353–368, Washington, D.C., August 2013. USENIX Association.

[5] Luke Dramko, Jeremy Lacomis, Edward J. Schwartz, et al. A taxonomy of C decompiler fidelity issues. In *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 379–396, Philadelphia, PA, August 2024. USENIX Association.

[6] Cheng Fu, Huili Chen, Haolan Liu, et al. Coda: An end-to-end neural program decompiler. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, pp. 3703–3714, 2019.

[7] Jordi Armengol-Estapé, Jackson Woodruff, Chris Cummins, et al. SLaDe: A portable small language model decompiler for optimized assembly. In *2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 67–80. IEEE, 2024.

[8] Peiwei Hu, Ruigang Liang, and Kai Chen. DeGPT: Optimizing decompiler output with LLM. In *31st Annual Network and Distributed System Security Symposium (NDSS 2024)*, San Diego, CA, USA, 2024. Internet Society.

[9] Ruchir Puri, David S. Kung, Geert Janssen, et al. CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks. In Joaquin Vanschoren and Sai-Kit Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, Vol. 1. Curran Associates, 2021.

© 2026 Information Processing Society of Japan

表 A·1　実験および初期集団生成の設定
Table A·1　Experimental and Seed-Generation Configuration.

項目	設定値
集団サイズ	256
評価回数上限	32,768 / trial
ルールベース修正	最大 3 回 / candidate
選択方法	lexicase, tournament
最適化レベル	O0, O2
コンパイラ	Clang 14.0.6, x86-64
コンパイルオプション	`-m64 -g -fno-pie -no-pie`
試行回数	3 / condition
条件数 / arm	72
試行数 / arm	216
LLM 生成候補の割合	0.5
候補生成 LLM	enhanced-llm4decompile-end-6.7b
temperature	0.2
生成トークン数上限	512
コンパイル修正 LLM	gpt-4.1-mini
候補データベース	Project CodeNet
候補検索	BM25 + edit-distance re-ranking

付　　録

A.1　実験設定の詳細

評価対象は，Project Euler の 6 課題（euler1–euler6）と，Learn C the Hard Way に由来する 12 課題（hw1，hw3，hw6–hw15）からなる 18 個の単一関数 C プログラムである．先行研究で扱われた 19 課題のうち，複数関数を含む hw18 は，現在の実装が単一関数を探索対象としているため除外した．また，先行研究で使用されたソースコードは公開されていないため，各評価対象は対応する演習または問題文に基づいて再実装したものであり，先行研究と課題は対応するが，ソースコードおよび生成されるバイナリの一致は保証されない．

表 A·1 に，実験で共通して用いた探索および初期集団生成の設定を示す．

データベース候補の取得では，ターゲット関数の命令列を検索情報として BM25 による候補検索を行い，編集距離に基づいて候補を再順位付けする．取得した候補の中から類似度を考慮して初期候補を選択し，ターゲット由来の C コードテンプレートへ関数本体を挿入する．

A.2　条件別の全実験結果

表 A·2 に，全 72 条件について，ベースラインおよび LLM-mixed seeding における初期最高類似度 S_0 と最終到達類似度 S_f を示す．各値は 3 回の独立した試行の平均であり，$\Delta = S_{f,\mathrm{LLM}} - S_{f,\mathrm{base}}$ は最終到達類似度の差をパーセントポイントで表す．本文 4.2 節におけるベンチマーク単位の集約値は，これらの条件別結果をもとに算出した．

A.3　RQ2 の補助分析

A.3.1　最終類似度差の単純分解

本文 4.3 節では，最終到達類似度の差を，初期最高類似度の差 Δ_{seed} と，初期化後の探索による改善量の差 Δ_{search} に分解した．**表 A·3** に，その集計結果を示す．ここで，$G = S_f - S_0$ は初期集団から探索終了時までの絶対的な改善量を表す．

この分解では，最終類似度の差の大部分が初期最高類似度の差として現れる．一方，LLM-mixed seeding では S_0 が高く，100%までに残された改善可能な幅が小さいため，絶対的な改善量 G のみを用いて後続探索の効果を比較することは適切ではない．このため，本文では残存差分縮小率を用いて初期類似度の違いを考慮した分析を行う．

A.3.2　残存差分縮小率の回帰分析

残存差分縮小率を目的変数として，初期最高類似度 S_0，最適化レベル，選択方法，および評価対象プログラムの影響を調整した線形回帰分析を行った．$S_0 = 100$ では残存差分縮小率を定義できないため，該当する LLM-mixed seeding の 15 試行を除外し，417 試行を分析対象とした．なお，本データには $95 \le S_0 < 100$ に該当する試行は存在しないため，$S_0 < 100$ を分析対象とする定義は 417 試行を一意に選択する．

表 A·4 に回帰分析の結果を示す．仕様 1 では初期類似度，最適化レベルおよび選択方法を説明変数とし，仕様 2 ではこれらに評価対象プログラムの固定効果を加える．

評価対象プログラムを固定効果として含めた仕様 2 では，LLM-mixed seeding の係数は+0.164 であった．また，評価対象プログラム単位でクラスタ化した標準誤差を用いた場合にも $p = 2.4 \times 10^{-3}$ であり，係数の正の関係は維持された．S_0 の係数が負であることは，初期類似度が高くなるほど残された改善余地が小さくなる影響と整合する．

A.4　初期集団のコンパイル可能性と多様性

LLM-mixed seeding による探索進捗の改善について，初期集団が少数の類似した候補へ集中した結果である可能性を確認するため，初期集団のコンパイル成功率と，コンパイル可能な候補に含まれる異なる適応度値および類似度値の数を調べた．**表 A·5** に結果を示す．

LLM-mixed seeding では，O0 および O2 のいずれにおいてもベースラインより高いコンパイル成功率が得られた．また，異なる適応度値および類似度値の数もベースラインより多かった．これらの値は候補間の意味的な多様性そのものを表すものではなく，進化的操作によって到達可能な探索領域を直接評価する指標でもない．したがって，本分析は初期集団の多様性を限定的に評価する補助指標と

© 2026 Information Processing Society of Japan

表 A·2　全 72 条件における初期最高類似度および最終到達類似度
Table A·2　Full Per-Condition Results for Initial and Final Similarity.

		O0					O2				
		Baseline		LLM-mixed		Δ	Baseline		LLM-mixed		Δ
Target	Selection	S_0	S_f	S_0	S_f	(pp)	S_0	S_f	S_0	S_f	(pp)
euler1	lexicase	50.00	59.70	100.00	100.00	+40.30	6.56	6.56	36.07	65.99	+59.43
	tournament	50.00	60.00	100.00	100.00	+40.00	6.56	9.35	54.70	97.74	+88.39
euler2	lexicase	52.78	59.49	69.04	88.80	+29.32	38.50	37.88	100.00	100.00	+62.12
	tournament	52.78	52.08	72.98	91.18	+39.09	38.50	37.88	100.00	100.00	+62.12
euler3	lexicase	58.54	63.80	76.74	86.38	+22.58	24.30	42.45	24.30	84.26	+41.81
	tournament	58.54	61.54	76.74	81.88	+20.34	24.30	22.94	24.30	56.17	+33.23
euler4	lexicase	49.41	64.08	50.57	77.69	+13.61	14.88	18.42	38.53	70.38	+51.95
	tournament	49.41	54.68	50.57	57.03	+2.35	14.88	17.54	34.61	36.86	+19.32
euler5	lexicase	47.46	61.63	47.46	75.67	+14.04	18.75	19.56	25.35	31.57	+12.01
	tournament	47.46	54.09	53.86	59.83	+5.74	18.75	18.03	25.35	22.43	+4.41
euler6	lexicase	46.05	57.65	53.97	93.10	+35.46	53.33	53.33	69.17	69.17	+15.83
	tournament	46.05	49.50	53.97	87.36	+37.85	53.33	53.33	82.50	82.50	+29.17
hw1	lexicase	0.00	80.00	86.96	86.96	+6.96	58.97	58.97	55.56	76.92	+17.95
	tournament	0.00	80.00	84.64	88.53	+8.53	58.97	58.97	53.70	61.11	+2.14
hw3	lexicase	46.51	74.29	51.43	78.52	+4.23	58.33	58.33	48.41	58.14	-0.19
	tournament	46.51	67.13	51.43	67.40	+0.27	58.33	58.33	48.41	48.41	-9.92
hw6	lexicase	30.51	37.78	29.73	36.88	-0.89	14.78	32.15	15.84	36.31	+4.16
	tournament	30.51	37.29	30.05	31.83	-5.46	14.78	28.69	19.50	28.95	+0.26
hw7	lexicase	37.33	62.55	36.52	57.52	-5.03	36.84	47.96	32.28	63.61	+15.65
	tournament	37.33	50.96	36.52	44.61	-6.35	36.84	38.10	34.56	39.00	+0.91
hw8	lexicase	33.66	43.58	33.66	45.12	+1.54	10.39	27.36	60.00	60.00	+32.64
	tournament	33.66	34.62	33.66	34.62	0.00	10.39	26.18	57.17	57.63	+31.44
hw9	lexicase	25.28	40.24	26.60	39.52	-0.72	25.58	49.19	31.77	60.67	+11.48
	tournament	25.28	30.79	26.60	33.05	+2.25	25.58	40.19	29.54	39.92	-0.27
hw10	lexicase	32.60	55.53	32.60	52.05	-3.48	25.20	37.04	49.13	71.00	+33.96
	tournament	32.60	47.74	32.60	49.49	+1.75	25.20	33.10	54.75	67.12	+34.03
hw11	lexicase	34.12	48.43	34.12	55.68	+7.25	12.14	32.61	46.32	68.12	+35.51
	tournament	34.12	50.97	34.12	48.28	-2.69	12.14	31.88	40.70	69.75	+37.86
hw12	lexicase	43.55	56.71	67.03	85.51	+28.79	29.09	32.35	34.92	53.54	+21.18
	tournament	43.55	48.69	70.33	79.71	+31.03	29.09	33.33	34.34	44.20	+10.87
hw13	lexicase	31.93	38.35	31.93	39.88	+1.53	5.84	13.21	13.33	14.00	+0.79
	tournament	31.93	35.17	31.93	35.17	0.00	5.84	11.73	11.11	12.33	+0.60
hw14	lexicase	0.00	74.07	94.87	100.00	+25.93	30.43	32.32	65.54	70.67	+38.34
	tournament	0.00	74.07	97.44	100.00	+25.93	30.43	32.32	65.98	72.88	+40.56
hw15	lexicase	16.75	30.46	16.75	33.71	+3.26	8.88	26.54	24.94	43.50	+16.95
	tournament	16.75	20.69	16.75	20.69	0.00	8.88	24.13	26.88	30.67	+6.54

表 A·3　最終類似度差の初期候補成分と探索成分への単純分解
Table A·3　Naive Decomposition of the Final-Similarity Gain.

条件	Δ	Δ_{seed}	Δ_{search}	$\bar{G}_{\text{base}}$	$\bar{G}_{\text{LLM}}$
全体	+17.90	+17.29	+0.60	+12.53	+13.13
O0	+11.81	+17.26	-5.44	+17.93	+12.49
O2	+23.98	+17.33	+6.65	+7.13	+13.78

して扱う．

A.5　LLM-Mixed Seeding の詳細

A.5.1　LLM への入力

LLM による初期候補生成は関数単位で行う．ターゲット関数の逆アセンブル結果を，LLM4Decompile と同様の形式を用いて次の prompt へ整形する．

```
# This is the assembly code:
<func_name>:
<one instruction per line>
# What is the source code?
```

表 A·4　残存差分縮小率に対する回帰分析
Table A·4　Regression Analysis of Residual Closure.

	仕様 1	仕様 2
LLM-mixed 係数	+0.120	+0.164
p（OLS）	4.6×10^{-7}	1.3×10^{-12}
p（条件単位 cluster）	4.1×10^{-4}	2.0×10^{-6}
p（ベンチマーク単位 cluster）	1.5×10^{-2}	2.4×10^{-3}
p（条件平均）	3.3×10^{-3}	4.3×10^{-5}
S_0 係数（1 pp 当たり）	-0.001	-0.004
O2 係数	-0.118	-0.148
試行数	417	
条件数	140	

© 2026 Information Processing Society of Japan

表 A·5　初期集団のコンパイル成功率と多様性指標
Table A·5　Generation-0 Compilability and Diversity.

条件	opt	Compile	Unique fitness	Unique similarity
Baseline	O0	41.1%	3.3	4.9
LLM-mixed	O0	57.5%	10.8	15.0
Baseline	O2	33.3%	1.9	2.7
LLM-mixed	O2	58.1%	12.8	18.0

prompt には，上記のアセンブリコードと関数名のみを含める．C コードのテンプレート，ターゲット関数のシグネチャ，ABI に関する情報，その他の自然言語によるヒントは与えない．各命令はプログラム順に並べ，Capstone が出力したニーモニックおよびオペランド文字列を使用する．また，LLM へ入力するのは正規化前の逆アセンブル結果であり，適応度計算および実験結果の評価に用いる正規化命令トークン列は入力しない．

A.5.2　生成結果の抽出

LLM の生成結果を C の translation unit として Tree-sitter で解析する．構文木から要求した関数名と完全に一致する関数定義を探索し，該当する関数が存在する場合には compound statement に相当する関数本体のみを抽出して，ターゲット由来のテンプレートへ挿入する．LLM が生成した関数シグネチャは使用しない．

対象関数名と一致する関数定義を取得できない出力は利用しない．具体的には，自然言語のみを含む出力，異なる関数名を持つ出力，関数定義を伴わない compound statement などは利用不能として扱い，対応するテンプレートのスタブ本体を維持する．

A.5.3　初期候補に対する LLM 修正

LLM によるコンパイルエラー修正は，LLM が生成した初期候補に対してのみ，進化的探索の開始前に適用する．まず，すべての候補に共通するルールベースのコンパイルエラー修正を行う．その後も LLM 生成候補がコンパイルできない場合には，gpt-4.1-mini を用いてコンパイルエラー修正を行う．

LLM による修正時には，現在の C ソースコード全体と，直前の Clang による標準エラー出力を入力する．system prompt には次の固定文を用いる．

```
You are a C compiler-error repair assistant. The user
supplies broken C source plus clang's stderr.
Reply with ONLY a JSON object of the form
{"corrected_source": "<the entire repaired C source>"}.
Do not include explanations, markdown fences, or any
text outside the JSON object.
```

返却された JSON から修正後のソースコードを取得して再コンパイルし，コンパイルに成功した最初の修正結果を採用する．設定された修正回数の範囲内でコンパイル可能な候補を得られなかった場合には，他のコンパイル不能候補と同様に最悪の適応度を与える．

この LLM 修正はデータベースから取得した初期候補には適用せず，進化的探索によって生成された候補にも適用しない．したがって，実験で評価する LLM-mixed seeding は，LLM による候補生成と，初期段階における LLM コンパイルエラー修正を合わせた初期化手法として解釈する必要がある．

© 2026 Information Processing Society of Japan